SCENARIO 1 | START THE INCIDENT

Material request outage

Purpose: show how a real application failure automatically reaches Azure SRE Agent.

1. Open the deployed demo

Material-request app / Azure SRE Agent / Azure resource group

1. Open the existing onboardinglab-agent; it is already configured for this demo. Its incident response plan is Autonomous: SRE investigates and applies only the approved NSG repair.
2. In Azure, confirm PostgreSQL is Ready. Open Monitor > Alerts; choose Last 7 days and Alert condition All. Find this demo's checkout-failures alert and check its condition is Resolved, not Fired. User response Closed is not sufficient.
3. In the app, click New request. Enter Screen panel, quantity 6, plant Helsinki. Submit once.
4. Find the returned request number in the saved list. Confirm its material, quantity and plant. If this fails, fix the baseline before continuing.

2. Inject the controlled fault

In PowerShell, paste this single line from any directory. The script contains the demo target parameters. Wait for 'Fault inject completed'. This blocks PostgreSQL access, not the web process.

```
& 'C:\Users\zhenzh\projects\metso\sre-agent\labs\onboardinglab\scripts\scm-fault.ps1' inject
```

3. Submit one request while the fault is active

1. Stay on Material requests. Click New request, enter Wear liner, quantity 4, plant Turku, then click Submit request once.
2. Wait for the submission failure message. This was a real database-write attempt; changing form values alone does not submit anything.

> **Expected result**
> One qualifying failed submission is enough for this alert rule. Azure Monitor and SRE can take minutes to process it. Simulation is not needed; do not keep retrying. If SRE has not repaired it within ten minutes of Inject, use the fallback reset on page 2.

SCENARIO 1 | INVESTIGATE AND RECOVER

Watch SRE repair the incident

Purpose: see evidence-backed diagnosis and the approved automatic repair, then save a new request.

4. Inspect the automatically created investigation

1. In onboardinglab-agent, click Incidents in the left sidebar. Open the newest [Sev2] checkout-failures investigation. Check the initial alert card's timestamp matches this run; an older thread must be labeled recorded.
2. Inspect the request/dependency telemetry and Azure checks. The finding should identify the outbound NSG rule blocking PostgreSQL TCP 5432, not just a generic database error. Do not tell SRE the answer.
3. Open scm-workiq-managed and expand Result, not Parameters. Verify the actual returned SCM-INC-017 incident-email source. A Completed badge alone is not proof of success.
4. Open Foundry IQ's Knowledge Base Retrieve (scm-foundry-iq). Check the retrieved SCM-Operations-Handbook.pdf sections and how SRE compares both sources with the live Azure evidence.

5. Check the automatic repair

In the same thread, inspect SRE's executed NSG update and its result. Confirm the rule is Allow. No approval or Reset button is needed; do not run the operator reset first and call that an SRE repair.

Fallback only: if repair fails or ten minutes elapse, stop any optional simulation and paste this line in PowerShell from any directory. It restores the rule, but does not start a separately stopped database.

```
& 'C:\Users\zhenzh\projects\metso\sre-agent\labs\onboardinglab\scripts\scm-fault.ps1' reset
```

6. Confirm recovery

1. Submit Filter cartridge, quantity 3, plant Tampere; then Bearing assembly, quantity 2, plant Helsinki. Verify both generated IDs and saved rows.
2. In the same SRE thread, provide the new request IDs and time. Ask it to verify fresh successful request/dependency telemetry without further resource changes.

> **Expected result**
>
> New rows and successful PostgreSQL dependencies confirm recovery. The alert can take roughly 10-15 minutes or longer to resolve; continue with the next scenarios instead of running another fault. Closed or Acknowledged is not the same as Resolved.

SCENARIOS 2 AND 3

Health check and Live Report

Purpose: show repeatable operational checks and a reusable view of application health.

Scenario 2: Run the existing health workflow once

1. In the SRE sidebar, open Automation. Find reservation-daily-health-report; its internal name is unchanged.
2. Select its checkbox and click Run now. Do not enable the recurring schedule.
3. Click the task name to open its executions, then click the newest Thread name link.
4. Read the actual findings: observation window, successful/failed requests, PostgreSQL dependencies and any risks or missing data.
5. Leave the task Off/Paused and its mode Review.

> **Expected result**
> The task thread contains findings grounded in the deployed app's telemetry. A Success badge alone only confirms handoff. Historical failures can remain in the observation window after recovery.

Scenario 3: Refresh the saved Live Report

Open Material Request Health

1. Open Material Request Health and click Refresh.
2. Inspect the request summary, request trend and outcome breakdown.
3. Inspect the PostgreSQL summary and trend. Find the incident and recovery intervals.
4. Check baseline coverage for data freshness, sparse history and evidence limitations.

> **Expected result**
> A refreshed operational view shows failures followed by recovery, with explicit time windows and coverage. Telemetry counts attempts, not only saved rows; two successful requests are not a load test.

SCENARIO 4 | CHANGE REVIEW AND CLEANUP

Compare a safe and risky change

Purpose: show SRE assessing the operational impact of a proposed code change.

Good change: zhenbzha/sre-agent#1

Risky change: zhenbzha/sre-agent#2

1. Open PR #1. Inspect the database deadline change from 5000 to 4500 milliseconds and its matching tests/documentation.
2. In SRE, open its completed ticketing-pr-validation review. Match the PR number and head commit, then inspect the PASS explanation.
3. Open PR #2. Inspect await client.end() before forced cleanup, then its SRE BLOCK explanation: hanging cleanup can hold the response and concurrency slot past the deadline.

> **Expected result**
>
> The actual review explains why one change passes and the other is risky. These are recorded reviews, not fresh executions. A green delivery job is not a PASS verdict; BLOCK is advisory, not a merge gate.

Optional: trigger fresh reviews

Close and reopen each PR. Follow GitHub Actions > SRE Agent PR validation, then the new ticketing-pr-validation threads in SRE. Match the head commits. Do not merge.

Before ending the demo

1. If simulation is running, click Stop simulation. Run Reset if any fault remains or the incident was interrupted; traffic stopping does not reset it.
2. Confirm a new request saves and appears in the list. Check PostgreSQL Ready and the fault rule Allow if recovery fails.
3. Leave the health schedule Off/Paused in Review. Keep the existing guarded incident response plan Autonomous. Check the alert's monitor condition; disclose if it is still resolving.

If the live investigation is incomplete

Inspect the existing alert thread before restarting anything: SRE can finish after a local runner stops. Work IQ InternalError permits at most one identical retry after 30 seconds, handled by SRE. Do not add manual searches and call them automatic.

Reset no later than ten minutes if investigation stalls. Clearly label older threads recorded. The 25 October rehearsal demonstrates investigation only; its operator safety reset is not proof of automatic recovery.

Keep the database private. Do not change permissions or upload this action guide and its fault instructions into SRE knowledge.